

PRACTICE I

Pablo Fernández (100523036)

and

Marcos Villamiel (100522963)

INDEX

Introduction.....	3
Phase 1: Instance Collection.....	4
Phase 2: Classification.....	6
Phase 3: Building an automatic agent.....	9
Phase 4: Prediction.....	11
Questions.....	14
Final Conclusion.....	16

INTRODUCTION

The objective of the Snake game is to consume food in the shortest time possible, maximizing the overall score. For this project, a provided simulator is used, allowing for automatic control of the Snake and data collection from the environment. This data includes variables such as food position, distance to food, the snake's body position, and game points, which can be leveraged to apply Machine Learning techniques, enabling the Snake to make autonomous decisions.

The main goal of this project is to predict the next movement of the Snake (a classification problem) and to estimate the score for the next turn based on the previous turn and the Snake's movement (a regression problem). To achieve this, a dataset of training examples must be generated, serving as input for classification and prediction models.



PHASE 1: INSTANCES COLLECTION

The data that has been collected by the feature extraction function from playing with the keyboard the Snake game consists of 2514 instances and 21 attributes. The attributes are the following:

- **snake_pos_x**: the x position of the head of the snake (Numeric)
- **snake_pos_y**: the y position of the head of the snake (Numeric)
- **snake_pos_x_next**: the x position of the head of the snake in the next tick (Numeric)
- **snake_pos_y_next**: the y position of the head of the snake in the next tick (Numeric)
- **food_pos_x**: the x position of the food (Numeric)
- **food_pos_y**: the y position of the food (Numeric)
- **food_side_y**: the position of the snake with respect to the food in the y axis (Nominal {UP, DOWN})
- **food_side_x**: the position of the snake with respect to the food in the x axis (Nominal {RIGHT, LEFT})
- **manhattan_distance**: the distance between the head of the snake and the food (Numeric)
- **next_manhattan_distance**: the distance between the head of the snake and the food in the next tick (Numerical)
- **large_snake**: the length of the snake (Numerical)
- **right_safe**: TRUE if the right position of the head of the snake is available, FALSE otherwise (Nominal {TRUE, FALSE})
- **left_safe**: TRUE if the left position of the head of the snake is available, FALSE otherwise (Nominal {TRUE, FALSE})
- **up_safe**: TRUE if the position above the head of the snake is available, FALSE otherwise (Nominal {TRUE, FALSE})
- **down_safe**: TRUE if the position below the head of the snake is available, FALSE otherwise (Nominal {TRUE, FALSE})
- **score**: the score in the actual tick (Numeric)
- **next_score**: the score in the next tick (Numeric)
- **cycle**: whether the snake is following the same directions in a cycle (Nominal {TRUE, FALSE})
- **body_indirection**: whether there is body of the snake between the head and the food (Nominal {TRUE, FALSE})
- **previous_direction**: the direction the snake took in the last tick (Nominal {UP, LEFT, DOWN, UP})
- **direction**: the direction the snake is taking (Nominal {UP, LEFT, DOWN, UP})

Future attributes have been calculated in different ways. The attributes `snake_pos_x_next` and `snake_pos_y_next` have been calculated by adding or subtracting the equivalent of one pixel in the board (10 units) according to the direction of the snake.

- If direction is 'UP' or 'DOWN', snake_pos_x_next will be the same as snake_pos_x while snake_pos_y_next will be snake_pos_y + 10, snake_pos_y - 10 respectively.
- If direction is 'RIGHT' or 'LEFT', snake_pos_y_next will be the same as snake_pos_y while snake_pos_x_next will be snake_pos_x + 10, snake_pos_x - 10 respectively.

In the case of next_manhattan_distance, it has been calculated the same way, but using the future x and y positions of the snake:

- $\text{abs}(\text{snake_pos_x_next} - \text{food_pos_x}) + \text{abs}(\text{snake_pos_y_next} - \text{food_pos_y})$

Finally, next_score by subtracting 1 unit to the previous score or adding 100 if the snake is in the position of the food:

- If next_snake_pos_x or next_snake_pos_y is not equal to food_pos_x or food_pos_y respectively, then next score will be score - 1.
- If snake_pos_x_next and next_snake_pos_y are equal to food_pos_x and food_pos_y respectively, then the next score will be score + 100.

PHASE 2: CLASSIFICATION

First, we will start with data preprocessing. To reduce dimensionality, we have identified some attributes that may be irrelevant for predicting the direction of the move. One attribute that does not provide significant information for this classification is the *score*, so we will remove it. Future attributes are also useless, so we will remove all of them (*snake_pos_x_next*, *snake_pos_y_next*, *next_manhattan_distance*, *next_score*). Now, we are left with 16 attributes with which we will do the prediction models. The data that has been collected comes from one single game.

ALGORITHMS

In order to find the best classification model we have decided to compare three different algorithms. The algorithms we have chosen for experimentation are ID3, J48 and LMT (Logistic Model Trees). We found LMT interesting because it combines decision trees with logistic regression models. ID3 does not interpret continuous attributes, so those attributes will necessarily be discretized so that this algorithm can be used. On the other hand, J48 can handle continuous and discrete attributes.

In general, these are three completely different and suitable algorithms for experimentation with different combinations of attributes to find the best model.

FIRST ITERATION

We have computed these first models without applying any filters. The only filter we have used is *discretize* to be able to compute the ID3 algorithm.

	ID3	J48	LMT
Training accuracy	0.977126	0.954252	0.930383
Test accuracy	0.972637	0.950249	0.91791

From this first iteration, we can conclude that ID3 is the best algorithm for the model with the selected data. However, we will try other combinations to find a better model.

SECOND ITERATION

In the second iteration, we have modified the training and test sets. We want to reduce the dimension of the data, so we want to obtain a subset of the original set that keeps enough information reducing the number of attributes. Applying the Attribute Evaluator *CorrelationAttributeEval* we can see that the attributes that are less correlated to the target variable are *cycle*, *manhattan_distance*, *snake_pos_y*, *body_indirection*, *snake_pos_x* and *large_snake*. We will remove them and apply the same algorithms with the 10 selected features.

	ID3	J48	LMT
Training accuracy	0.957235	0.925908	0.924913
Test accuracy	0.942786	0.912935	0.915423

We can observe how the accuracy of the models has decreased, but not that much. Now the models are more flexible and less biased. Again, discretizing the numerical attributes to apply the ID3 algorithm has been the best approach because it is the most accurate model.

THIRD ITERATION

For the third iteration, we have collected data from 5 different games. We have now used this entire dataset as the training set (4596 instances), and then we have played another game. This last game's dataset will be the test set (1640 instances). We have applied the same preprocessing as in iteration 2 for J48 and LMT. However, for ID3 there was some unbalanced data that was causing overfitting. These were the variables *food_pos_x* and *food_pos_y*, so in order to keep reducing dimensionality and mitigating this problem we decided to remove them. The results were the following:

	ID3	J48	LMT
Training accuracy	0.959095	0.9604	0.966493
Test accuracy	0.962805	0.959756	0.953049

It is clearly seen that collecting the data from different games is better for training the model. The results from this iteration have been similar for the three algorithms. However there is one algorithm that has been a little bit better than the others.

CONCLUSION

As a conclusion, the best model we have obtained is the ID3 algorithm during the third iteration. It is the one that has the least number of attributes (8) and has also given great results. Another conclusion we get is that the quality of the collected data is very good because the precision of the data is very high even with few variables.

PHASE 3: BUILDING AN AUTOMATIC AGENT

INTRODUCTION

The objective of this phase is to develop an automatic agent and compare its performance with those from previous tutorials. The automatic agent is built using models generated by Weka. Several packages were downloaded, and the Weka class was imported into the Snake game script. The `weka.predict` function is used to classify states and determine the optimal direction for the agent. This function takes the model name, the trained data, and the input data to predict the next move. Consequently, at each game tick, the agent makes a decision on which direction to take.

INITIAL MODEL IMPLEMENTATION

To begin, we used the best models obtained in Phase 2, as they had the highest accuracy. However, we soon realized that accuracy alone was not a strong determinant of the agent's actual in-game performance.

Our best model from Phase 2 was an ID3 model. Unfortunately, it could not be used because it was not included in the Weka class. As for LTM and J48 models with the previously selected variables, J48 performed better. In some cases, the agent was able to consume around five fruits. However, the LTM model exhibited a major flaw: once a direction was predicted, it remained fixed, preventing the agent from adjusting its movement dynamically.

MODEL OPTIMIZATION

To improve performance, we generated new models through multiple iterations. We discovered that models trained exclusively on categorical variables yielded the best results. The dataset used was *"training_keyboard_1_cat.arff"*.

We proceeded with new J48 and LTM models, selecting only the most decisive categorical variables for the snake's movement: *food_side_x*, *food_side_y*, *right_safe*, *left_safe*, *up_safe*, *down_safe*, and *body_indirection*. We deliberately removed *prev_direction* as it appeared to introduce poor decision-making in the game. Despite these refinements, the J48 algorithm did not outperform the previous model.

On the other hand, LTM (*"modelcatLTM.model"*) achieved a performance score of approximately 5000. This agent does not necessarily follow the shortest path to the fruit. Instead, it navigates towards the game's borders and moves along them until it aligns with the apple's x-coordinate, then adjusts its movement accordingly. When reaching the edge, it determines whether to move left or right. While this is not the most optimal path, it prevents the snake from crashing into walls. The primary reason for failure occurs when the snake's body grows significantly and it

randomly moves left or right in the middle of the screen, ultimately colliding with a wall. However, in terms of body length, the agent successfully avoids self-collisions.

PERFORMANCE COMPARISON

Compared to previous tutorials, our agent has successfully matched the performance achieved in the move_tutorial. However, it is still far from reaching the score attainable through manual keyboard play. Of course, this depends on the game's difficulty level, as both the agent and the move_tutorial are unaffected by difficulty settings.

CONCLUSION

In this phase, we successfully developed an automatic agent using Weka-generated models and optimized it for better in-game performance. Although it does not follow the shortest path, it demonstrates intelligent movement strategies that prevent premature failure. While the agent does not outperform human gameplay, it achieves a competitive performance level comparable to previous automated approaches.

	MOVE_TUTORIAL	AUTOMATIC AGENT	MOVE_KEYBOARD
Aproximate Score	Around 5000	Around 5000	Around 7000

PHASE 4: PREDICTION

As in phase 2, we will start with the preprocessing. Now, we are interested in predicting the variable *next_score*, which is numeric. Hence, we will apply regression models. Regression models take as inputs numeric variables, so we will start our preprocessing by eliminating all the nominal features, as we do not have in consideration to create dummy variables due to their irrelevance. After this preprocessing, we are left with 11 variables: *snake_pos_x*, *snake_pos_y*, *snake_pos_x_next*, *snake_pos_y_next*, *food_pos_x*, *food_pos_y*, *manhattan_distance*, *next_manhattan_distance*, *large_snake*, *score* and the target variable *next_score*.

ALGORITHMS

The algorithms that we will try for this experiment are M5P, RandomForest and DecisionStump. M5P is a decision tree algorithm for regression that builds linear models at its leaves, combining tree simplicity with linear regression capabilities. RandomForest is an ensemble method that constructs multiple decision trees and averages their predictions, improving accuracy through collective learning. DecisionStump, the simplest of the three, is a one-level decision tree that makes predictions based on a single feature split, serving as a weak but fast baseline learner. Each algorithm operates differently: M5P refines predictions with linear functions, RandomForest reduces overfitting via tree diversity, and DecisionStump offers minimal complexity. These methods represent varying levels of sophistication in regression tasks, and for that reason we have chosen them.

FIRST ITERATION

In this first iteration we have used all the variables numeric variables from the dataset and these are the results on predicting the target variable:

	MP5	RandomForest	DecisionStump
Correlation coefficient (r)	0.9974	0.9997	0.8587
Mean absolute error (MAE)	10.5831	7.2121	216.407
Mean squared error (MSE)	35.8491	12.3939	255.6509
Relative absolute error	2.4702 %	1.6834 %	50.5113 %
Relative squared error	7.198 %	2.4885 %	51.3311 %

From the output metrics, we can get clear conclusions. Decision Stump has given poor results for this problem, the very high MSE means that it makes a lot of mistakes, so we will discard it. Apart from that, the correlation coefficient is lower, so it loses more information than the other algorithms. Now, the decision of the winning algorithm is between MP5 and Random Forest. Random Forest has given better results and lower metrics. However, as we know, it is a more black-box algorithm, less feasible and more complicated to understand than MP5. If our goal was to find a model that explained properly the conclusions with an acceptable rate of error, then that would be our model. Since this is not the objective and we are looking for accuracy, the best algorithm for this model is Random Forest.

SECOND ITERATION

Now we will do more preprocessing. We have applied the Attribute Evaluator *ClassifierAttributeEval* for all the classifiers and with the Search Method *Ranker*. Since DecisionStump has done a very bad performance, we will directly remove it from the experiment.

For M5rules the best attributes are *score*, *large_snake*, *food_pos_x*, *food_pos_y*, *snake_pos_y*, *snake_pos_y_next*, *snake_pos_x*, *snake_pos_x_next*. Since the other variables are clearly less important, we will eliminate them.

For RandomForest the best attributes are *score*, *large_snake*, *food_pos_x*, *food_pos_y*, *snake_pos_y*, *snake_pos_y_next*, *snake_pos_x_next*, *snake_pos_x* and *body_indirection*.

We are conscious that by eliminating certain attributes we are losing some information, but according to the output of the attribute evaluator the information we are losing is very small in comparison to the information the other attributes provide. Now our attribute will be more sensitive to outliers, hence more flexible. We will do the regression models with these attributes for each algorithm.

	MP5	RandomForest
Correlation coefficient (r)	0.9957	0.9986
Mean absolute error (MAE)	4.747	6.6641
Mean squared error (MSE)	15.6281	8.8754
Relative absolute error	2.0738 %	2.9113 %
Relative squared error	5.6752 %	3.223 %

Again, Random Forest has done better over MP5. The correlation coefficient of both algorithms has slightly decreased, but it remains very high. In the MAE, MP5 has decreased significantly while Random Forest remains approximately equal. The MSE has decreased in both, but in MP5 more significantly. With respect to the relative errors, RAE and RRSE worsened in Random Forest but improved in M5P.

CONCLUSION

Overall, RandomForest seems to be the best algorithm because of its consistently high accuracy (near-perfect correlation, lowest errors) and robustness after feature selection. While M5P offers interpretability, its performance declined significantly with fewer features. A conclusion we get from this is that complex models like RandomForest handle refined feature sets better, whereas simpler models require careful feature engineering. For optimal results, we will use RandomForest. For transparency, we will use M5P.

QUESTIONS

1. What is the difference between learning these models with instances coming from a human-controlled agent and an automatic one?

The difference between learning these models with instances coming from a human-controlled agent and an automatic one is that a human-controlled takes actions freely without limitations whereas an automatic agent follows a predefined set of rules. The data collected in a human-controlled system will contain more diverse and human-like strategies while the automatic agent will be more consistent and optimized for specific objectives.

Learning from instances coming from a human-controlled agent will make the model more flexible but with some more noise in the data that may lead to mistakes in the prediction. On the other hand, the model that learns from instances based on fixed rules will be more prone to overfitting due to its lack of adaptability to unseen scenarios.

2. If you wanted to transform the regression task into classification, what would you have to do? What do you think could be the practical application of predicting the score?

To transform a regression task into a classification task, it could be separated into intervals the score: more or equal than X , or less than X . But we think this manner is not very useful to the aim of the task, we would make two categories, good or bad decision taking in account the score. This category uses the prediction of score and the Manhattan distance to see if the distance was reduced when the score was less than previous (good movement). So a classification is achieved, transforming the regression task.

3. What are the advantages of predicting the score over classifying the action? Justify your answer.

Predicting the score instead of classifying the action can offer greater flexibility and long-term optimization, as it allows the model to evaluate the potential outcome of each possible move instead of choosing one based on past behavior. This approach enables the agent to simulate or estimate the expected reward of different actions and select the one that leads to the highest predicted score, which aligns more closely with the goal of maximizing the overall game performance. Unlike action classification, which may overfit to suboptimal strategies present in the training data, predicting scores encourages better generalization and planning, especially in unseen or complex situations where the best move depends on future consequences rather than immediate patterns.

4. Do you think that some improvement in the ranking could be achieved by incorporating an attribute that would indicate whether the score at the current time has dropped?

The answer would be the same as mentioned earlier in question 2. This new category, considering the score, could indicate whether the movement made is good (brings the snake closer to the fruit) or a bad move (moves it away without improving the score).

FINAL CONCLUSION

This work consists of integrating all phases of machine learning covered in the tutorials of this course to develop an automatic agent and predictive models. For this, various theoretical tools (different algorithms used) and techniques (Weka, Python, Ubuntu, etc.) have been employed.

This project also demonstrates how theoretical concepts often differ significantly from practical implementation. Therefore, an engineering approach is necessary to iterate and refine the process multiple times, reflecting on possible improvements (as seen in Phase 3, where models with lower accuracy were chosen because they included variables that the agent could interpret more effectively).

Another important aspect to highlight is the significance of data collection and processing. We consider that the quality of the data collected at the beginning is crucial for the proper functioning of both our agent and the predictive models.